

TKS Architecture Module 7 — Governance Rails That Cannot Be Bypassed

Generated: 2026-01-04 08:42 UTC

This module hardens the system so it cannot “optimize itself into danger.” It defines **non-bypassable rails** that sit above Router/Executor/RPM and below the user interface, with special strength in **High-Stakes** and **Critical** modes.

1) Purpose

Your architecture explicitly optimizes for **efficiency/power/control** (Foundation 5) while aligning to a **highest desire** (Foundation 1). Governance rails ensure:

- F1 constraints remain dominant and enforceable
- High-Stakes policy cannot be bypassed by clever phrasing or internal loops
- Irreversible actions require formal clearance
- Self-modification is bounded and audited
- Tool usage is sandboxed and permissioned
- The system fails safe (PAUSE/BLOCK) when uncertain

2) Where governance sits (layering)

User/External Inputs

- *Policy Gate (this module)*
- RPM Planner
- Router (Module 1)
- Executor (Module 1)
- Verifier (Module 2)
- Memory + Scoring + Rumination (Modules 3–4)
- Logs + Replay (Modules 5–6)

Key rule:

> Governance rails run **before** actions and **after** verification. If rails disagree with downstream modules, rails win.

3) Non-bypassable gates (hard rules)

3.1 High-Stakes enforcement gate

Use your HS scoring rule (already trained):

- $HS \geq 0.45 \rightarrow$ High-Stakes (PAUSE)
- $HS \geq 0.70 \rightarrow$ Critical (BLOCK)

Hard rules:

- **Critical:** no execution of risky tool actions until verifier PASS + clearance
- **High-Stakes:** must perform ASK $\rightarrow$ SLOW $\rightarrow$ VERIFY before tool execution
- Any missing evidence in High-Stakes/Critical $\rightarrow$ PAUSE/BLOCK automatically

Canonical TKS expressions:

- High-Stakes: $(1:2:5^*):(ASK:SLOW:VERIFY)+GATE(PAUSE)$
- Critical: $(1:2:5^*):(ASK:SLOW:VERIFY)-EXECUTE$

3.2 Irreversibility gate

Mark actions as:

- reversible
- partially reversible
- irreversible

Hard rule:

- If `irreversible_action=true` then:
- require Verifier PASS
- require explicit “clearance token” (Section 4)
- require evidence binding
- log as a protected event (Module 6)

Examples of irreversible actions:

- deleting data

- transferring funds
- changing security settings
- publishing sensitive outputs

3.3 Permission gate (capabilities + sandbox)

Every tool call must pass:

- capability threshold (Module 3)
- permission checks:
 - file read/write scope
 - code execution sandbox
 - network allowed/denied
 - allowlist sources

Hard rules:

- No network unless explicitly enabled
- No filesystem deletion unless clearance token
- Code execution must be sandboxed
- Tools return evidence IDs by default

3.4 Anti-recursion / anti-runaway gate

Prevent infinite trees and obsessive loops.

Hard caps (defaults; tune later):

- RPM depth cap: 3 (Normal), 2 (High-Stakes initial), expandable to 3 only if uncertainty drops
- RPM breadth cap per node: 5
- Tool call cap per episode: 12 (Normal), 8 (High-Stakes), 0 (Critical until verified)
- Rumination budget per cycle: fixed (Module 0 spec)

Hard rule:

- If uncertainty does not drop after 2 expansions → PAUSE and ASK for missing info.

3.5 Self-modification gate (policy/template edits)

Any change to:

- router rules
- verifier thresholds

- reward weights
- promotion logic
- operator semantics
- canonical definitions

...is considered **self-modification**.

Hard rules:

- No self-modification in Critical mode
- High-Stakes self-modification requires:
 - replay harness pass (Module 5)
 - audit log entry (Module 6)
 - rollback plan and do-not-apply guard
- Self-modification must be versioned and reversible

4) Clearance tokens (formal permission to proceed)

A clearance token is a structured approval that can come from:

- a trusted user confirmation
- an internal “verified safe” proof (replay + evidence)
- an admin policy layer (in deployment)

4.1 ClearanceToken schema

```
{
  "token_id": "ct_00001",
  "scope": "file_delete|security_change|money_transfer|publish",
  "episode_id": "ep_...",
  "approved_by": "user|admin|verifier",
  "evidence_ids": ["ev_..."],
  "expires_at": 1735969999,
  "constraints": {
    "target": "path/account/system",
    "max_amount": 0,
    "dry_run_required": true
  }
}
```

Hard rule:

- If token missing or expired → BLOCK for irreversible action.

5) Alignment rail (F1 dominance)

Even if Foundation 5 pushes for efficiency/power, F1 is dominant.

Implement alignment dominance as:

- a hard constraint threshold $A_{\min}$ (e.g., 0.80)
- plus a penalty exponent in HS scoring (already in your HS formula via A^2)

Hard rules:

- If alignment score drops below $A_{\min}$ → PAUSE and switch to 1:2: * stack until resolved
- If request violates core constraints → refuse or require user clarification

6) Safe failure modes (how the agent should fail)

When rails block:

- do not “guess”
- do not “execute anyway”
- do not “work around”

Allowed outputs when blocked:

- ASK questions
- show VERIFY checklist
- request evidence or confirmation
- propose a dry-run/simulation plan
- provide a reversible alternative

7) Governance event logging (must be recorded)

Every gate decision must log:

- HS score and mode
- which gate triggered (irreversible, permission, recursion, self-mod)
- what evidence was missing
- what the user can do next
- policy versions (Module 6)

This is required for replay, auditing, and rollbacks.

8) Minimal implementation checklist

- Central Policy Gate module called before Router/Executor
- HS enforcement + irreversible action guard
- Permission + sandbox enforcement per tool
- Recursion/runaway caps + “uncertainty must drop” rule
- Self-modification manager: versioning + replay + rollback
- Clearance token system (schema + expiry + scope)
- Audit logging for every gate decision

End of Module 7.